Remembering Why, Answering Adaptively: Repository-Level Question Answering by Learning from Development Traces

ANONYMOUS AUTHOR(S)

Repository-level question answering is a fundamental activity in software development, supporting developers in understanding code, reasoning about design choices, and completing development tasks. Its importance has grown further with the emergence of human–AI collaborative development. However, existing approaches primarily rely on current repository snapshots, which may omit historical decision evidence, and lack awareness of developers' cognitive needs. These limitations hinder both answer correctness and cognitive fit. We propose **DevHound**, a repository-level question-answering approach that continuously observes and learns from development processes. DevHound transforms IDE operations and interactions with AI coding agents into development traces and constructs two complementary memories: Rationale Memory preserves historical decision rationale and its supporting evidence through task-level extraction and validation, while Developer Cognition Memory estimates developers' familiarity with individual knowledge points from behavioral evidence. At query time, DevHound refines questions using recent development context, retrieves and verifies evidence from the repository and Rationale Memory, and adapts explanation depth to developers' cognitive needs. We evaluate DevHound on 200 questions derived from an eight-week naturalistic study involving 10 developers and 10 repositories. Results show that DevHound improves answer correctness by 2.34–2.79 points on a 10-point scale over same-model repository-level QA baselines, with the largest gains on Process-based Questions. It also improves all five dimensions of cognitive fit and reliably constructs Rationale Memory across models. These findings suggest that continuously learning from development processes can turn development experience into reusable knowledge, supporting developers' understanding and decision-making in human–AI collaborative development.

Additional Key Words and Phrases: Repository-level question answering, development-process knowledge, evidence validation, developer cognition, coding agents

ACM Reference Format:
Anonymous Author(s). 2027. Remembering Why, Answering Adaptively: Repository-Level Question Answering by Learning from Development Traces. In *Proceedings of ACM International Conference on the Foundations of Software Engineering (FSE 2027).* ACM, New York, NY, USA, 21 pages.

1 Introduction

Answering questions about software repositories has long been a fundamental and important task [1, 20, 39, 41], and its importance has further increased under the emerging paradigm of human–AI collaborative development [13, 40]. Recent research [49] has shown that development knowledge Q&A constitutes a substantial proportion of software-development-related user–LLM dialogues, exceeding tasks such as code generation and program repair. With the rapid advancement of large language models, there has therefore been growing interest in LLM agents that can provide end-to-end support for Repository Q&A over software repositories.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
FSE 2027, Shenzhen, China
© 2027 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Repository Q&A should first ensure that developers' questions are answered correctly. Based on whether a question can be correctly answered by understanding only the current repository snapshot, repository questions can be categorized into two types: (1) **Snapshot-based Question**: All key information required to answer this type of question is contained in the current repository snapshot. A complete answer can be obtained solely by retrieving and understanding repository artifacts such as source code, configuration files, documentation, and tests. These questions typically concern what the system is or how it works. (2) **Process-based Question**: At least one piece of key information required to answer this type of question cannot be obtained from the current repository snapshot. Therefore, in addition to analyzing the current repository, answering such questions requires observing and understanding the development process and obtaining relevant evidence from it. These questions typically concern design rationale, trade-offs, failed attempts, or validation processes, and often take the form of "why" questions.

Beyond correctness, an effective Q&A system should also present answers in a way that developers can readily understand. Developers' familiarity with different areas of knowledge varies with their educational and professional backgrounds, as well as the development tasks they have recently worked on. Consequently, the same key point may require different levels of explanation for different developers [6, 17]. We refer to these varying explanatory requirements as developers' **cognitive needs**.

Therefore, effective Repository Q&A approach should achieve both **answer correctness** and **cognitive fit**. However, existing Repository Q&A approaches still have two major limitations that prevent them from fully realizing two aspects of value. (1) **Existing approaches lack the development-process evidence required to answer Process-based Questions**. Current Q&A agents mainly rely on repository snapshots for retrieval and reasoning. As a result, for questions that require understanding the development process, such as those involving design rationale or trade-offs, they often lack sufficient evidence to provide accurate answers. (2)**Existing approaches lack awareness of developers' cognitive needs**. Developers differ in their familiarity with the knowledge involved in a question, and therefore require different explanation focuses and levels of detail. However, existing systems typically cannot capture these differences and instead adopt a uniform strategy for organizing and presenting answers. As a result, they may over-explain knowledge that developers already understand while providing insufficient explanation for knowledge they are unfamiliar with, thereby increasing comprehension effort and reducing the practical usefulness of the answers.

These two limitations share a common root cause: existing Repository Q&A approaches typically observe only the outcome of development, i.e., the current repository snapshot, while lacking continuous observation and learning of the development process. In fact, the development process contains two types of information that are crucial for Repository Q&A. The first is **rationale evidence**, such as task requirement and design choice, which can help explain why the software evolved into its current state. The second is **cognitive evidence** left by developers during activities such as reading, modifying, seeking help, and asking follow-up questions, which can reflect their evolving understanding of different knowledge [11] and help the system determine what needs to be explained and to what depth.

Based on this insight, we propose DevHound, a repository-level Q&A approach that continuously observes and learns from the development process to recover the historical decision rationale needed for problem solving and to capture developers' cognitive needs. DevHound constructs two complementary types of memory from the development process. **Rationale Memory** stores rationale evidence grounded in development-process evidence, supplementing process knowledge that is missing from the current repository snapshot, such as design rationale, trade-offs, and validation evidence. **Cognition Memory** accumulates evidence from developers' development trajectories

that reflects their cognitive states, and uses it to estimate their explanation needs for different key points.

To evaluate DevHound, we conducted an 8 week naturalistic study involving 10 developers and 10 real-world repositories, and assessed it from three aspects: answer correctness, cognitive fit, and Rationale Memory construction. The results show that DevHound significantly improves the correctness of repository-level Q&A, with particularly strong gains on Process-based Questions. Its answers outperform the baseline across all five dimensions of cognitive fit and accurately identify developers' cognitive levels. Meanwhile, DevHound can construct Rationale Memory with high accuracy and acceptable cost.

2 Motivation

To more concretely illustrate the motivation of this study, we use a real development question from the Kubernetes community (Figure 1) to demonstrate the limitations of existing approaches to Repository Q&A.

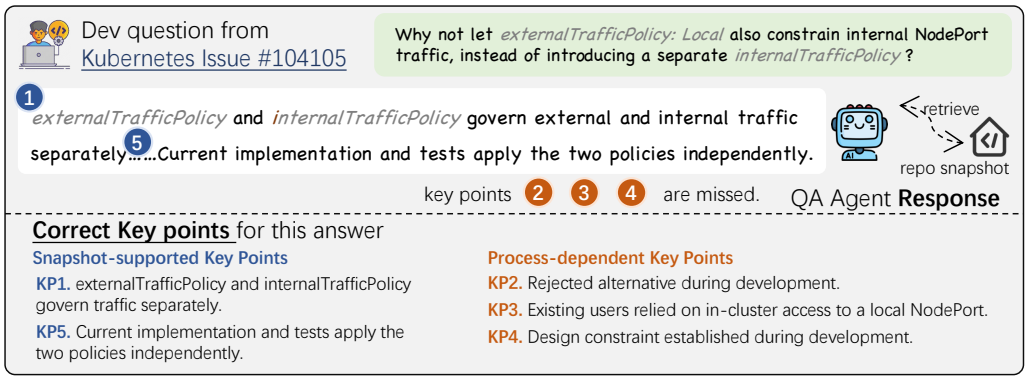


Fig. 1. Motivating example from Kubernetes Issue #104105

Given this developer question, existing approaches explore and understand the current repository, retrieve relevant code and configuration snippets, and generate an answer. As illustrated in Figure 1, this answer covers only **KP1 and KP5**, two of the five key points in the correct answer. The remaining **KP2–KP4** require development-process evidence absent from the current snapshot. For example, KP2 records a decision during solution exploration to reject treating all NodePort traffic as external, because this could drop internal requests when no local endpoint exists. KP3 records that existing users relied on in-cluster access to their local NodePort, making a change to this behavior a compatibility risk. KP4 captures the decision to support local routing for external traffic together with cluster-wide routing for internal traffic, which the maintainer discussion identifies as a reason for independent policies. Without such process evidence, these approaches cannot recover the remaining three key points. Inferring a plausible rationale from the implementation would not close this gap: the same behavior can reflect different motivations, and a credible explanation must establish which considerations actually guided the historical decision.

Therefore, the key limitation is not that existing approaches fail to retrieve a particular code file, but that **their problem-solving space lacks development-process knowledge that is no longer preserved in the current repository**. This observation directly motivates DevHound: the current repository typically preserves only the final outcomes of development decisions, rather than the complete process through which those decisions were formed. To address this limitation, DevHound continuously learns from the development process composed of problem reporting,

solution exploration, code modification, and result validation, distills evidence-supported historical decisions into Rationale Memory, and replays relevant decision trajectories during problem solving to better answer developers' questions.

3 Approach

DevHound supports repository-level Q&A by learning from development traces (Figure 2). It builds Rationale Memory to preserve evidence-backed process knowledge absent from the current repository, and Cognition Memory to estimate knowledge-point familiarity solely to regulate explanation depth. At query time, it refines questions with task context, reasons over repository and rationale evidence, and adapts explanations to developers' needs. The following subsections describe the Q&A pipeline and memory construction.

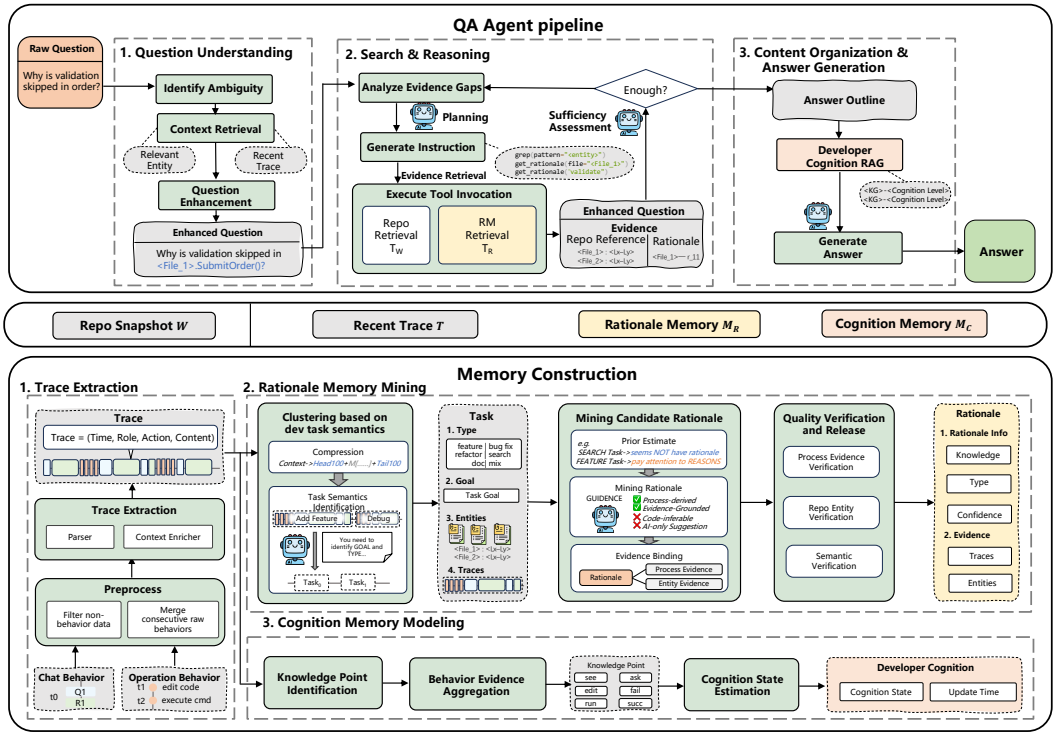


Fig. 2. Overview of DevHound: repository Q&A grounded in development-process evidence and developer cognitive states, together with the construction of two types of memory.

3.1 QA Agent Pipeline

DevHound's repository Q&A pipeline consists of three stages: **(1) Question Understanding**, **(2) Search & Reasoning**, and **(3) Content Organization & Answer Generation**.

(1) Question Understanding. Developers' original questions may be imprecise [15, 51], and question quality can affect answer correctness [29, 43]. DevHound therefore introduces a behavior-aware stage for question understanding before search and reasoning. Because entity accuracy and task-context clarity are critical to subsequent search and reasoning, the system focuses on these two aspects.

Specifically, given a developer's question Q, DevHound first identifies potentially ambiguous entities and underspecified task context. It then retrieves contextual information from recent Trace T and entities in the code repository W, obtaining $C_Q = \mathrm{R}(Q; T, W)$. Given this candidate context, DevHound enhances the original question as $Q' = \mathrm{E}(Q, C_Q)$. The enhanced question explicitly captures the task context and intent that were implicit in the original question and refers to entities more clearly and precisely, thereby providing a reliable input for subsequent search and reasoning.

(2) Search & Reasoning. Given the enhanced question Q', DevHound iteratively searches and reasons over the problem-solving space $S = W \cup M_R$, comprising the repository snapshot W and Rationale Memory M_R. The loop terminates when the accumulated evidence is sufficient to answer Q', the maximum number of rounds is reached, or the system determines that the question cannot be answered.

DevHound's complete problem-solving space S consists of the given repository snapshot W and Rationale Memory M_R: $S = W \cup M_R$. In round t, the system performs planning, retrieval, and verification in sequence.

(1) **Planning.** The planner determines the additional evidence needed and generates a tool-use plan for the current round, $P_t = \mathrm{Plan}(Q', E_{t-1})$. It first classifies the question as Snapshot-based or Process-based. For Snapshot-based Questions, it retrieves only from the repository snapshot W; for Process-based Questions, it retrieves from both W and Rationale Memory M_R to recover development-process evidence absent from the current repository.

(2) **Evidence Retrieval.** The retrieval stage executes P_t and obtains $R_t = \mathrm{Retrieve}(P_t; S)$. The tool set $T = T_W \cup T_R$ consists of repository-retrieval tools T_W (e.g., `ls`, `grep`, and `read`) and Rationale Memory tools T_R (e.g., `list`, `get`, and `search`). The retrieved results are added to the accumulated evidence: $E_t = E_{t-1} \cup R_t$.

(3) **Sufficiency Assessment.** The verifier evaluates whether E_t is sufficient to answer Q', i.e., $V_t = \mathrm{Verify}(Q', E_t)$. If sufficient, it emits `finish` and proceeds to answer generation; otherwise, another planning–retrieval–verification round begins. If sufficient evidence is still unavailable after the maximum T rounds, the problem is marked as unanswerable.

(3) Content Organization & Answer Generation. This stage first organizes the answer content based on the evidence obtained through search and reasoning, producing an answer outline that preserves correctness. It then adjusts the explanation depth for each knowledge point according to the developer's current cognitive state, fills in the details, and generates the final answer.

Specifically, DevHound first generates an answer outline from the evidence obtained during search and reasoning. The outline is a set of knowledge points that the answer must cover: $K = \{k_1, k_2, \ldots, k_n\}$. The system then retrieves from Cognition Memory M_C the cognitive-state value $c_{u,i}$ of developer u for each knowledge point k_i. A higher value indicates that the developer is more likely to be familiar with that knowledge point. Based on this value, the system assigns the developer a level for the knowledge point:

$$\mathrm{Level}(u, k_i) = \begin{cases} \text{expert}, & c_{u,i} \geq \tau_2, \\ \text{regular}, & \tau_1 < c_{u,i} < \tau_2, \\ \text{novice}, & c_{u,i} \leq \tau_1, \end{cases}$$

where $\tau_1 < 0.5 < \tau_2$. The novice, regular, and expert levels describe only the developer's familiarity with the current knowledge point; they are not fixed labels of the developer's overall ability. This three-level scale follows the central insight of the expertise reversal effect [17]: the same detailed explanation may help a developer who is unfamiliar with a knowledge point while imposing redundant cognitive load on one who is familiar with it. Accordingly, when elaborating the

answer, the novice level includes the underlying concepts and reasoning process, the regular level explains the implementation mechanism, and the expert level retains the conclusion, applicability boundaries, and evidence while compressing familiar background information.

3.2 Constructing Memory by Observing and Learning from the Development Process

3.2.1 ***Trace Extraction.*** Software development is mediated by programming environments, and developer activities can be observed in IDEs [18] and AI coding agents [21]. DevHound therefore observes developer activities in these two environments, obtaining **Operation Behavior** in the IDE, $\mathcal{B}^{\text{ide}} = \left\{b_1^{\text{ide}}, b_2^{\text{ide}}, \ldots, b_p^{\text{ide}}\right\}$, and **Chat Behavior** in the AI coding agent, $\mathcal{B}^{\text{agent}} = \left\{b_1^{\text{agent}}, b_2^{\text{agent}}, \ldots, b_q^{\text{agent}}\right\}$.

Raw events produced by different programming environments and tools vary substantially in data structure and semantic representation. For example, agent interaction logs are typically message sequences containing fields such as role and content, whereas terminal operations in an IDE mainly contain commands, outputs, and execution states. These heterogeneous events cannot be used directly in a unified downstream analysis. We therefore normalize developer behavior B into a unified data structure, **Trace**:

$$Trace = \langle T, R, A, C \rangle$$

where T denotes the timestamp, R the role, A the action, and C the content.

Trace generation comprises **Preprocessing** and **Trace Extraction**:

(1) **Preprocessing.** DevHound filters out actions not initiated by the developer and initially aggregates consecutive actions.
(2) **Trace Extraction.** For each event e_i from source s_i, parser $\mathcal{P}_{s_i}$ extracts fields such as time and role and processes the context according to the source type.

DevHound thus converts IDE operations and human–agent conversations into a structurally consistent, chronological, and traceable Trace sequence for task identification, Rationale Memory construction, and Cognition Memory updates.

3.2.2 ***Rationale Memory Construction.*** Trace provides fine-grained process data for Rationale extraction, but extracting Rationale directly from it poses three challenges. (1) The evidence boundary of a Rationale is difficult to determine. Continuous Trace may span multiple development tasks, and inappropriate context segmentation can split related evidence or mix unrelated activities. (2) Rationale signals are sparse and implicit. Design intent, implicit contracts, risks, external dependencies, and reusable workflows are scattered across a small fraction of Trace and must be distinguished from routine operations and noise. (3) The trustworthiness of a Rationale is difficult to ensure. Unsupported or hallucinated Rationale may mislead downstream Q&A, requiring validation against development-process evidence and repository entities.

To address these challenges, DevHound does not learn and extract Rationale directly from the complete Trace sequence. Instead, it constructs Rationale Memory through a three-stage pipeline. The details are as follows.

(1) Clustering by Development-Task Semantics

DevHound first clusters Trace into Tasks according to task semantics. A Task is a set of semantically related Trace that are temporally contiguous and concern the same development goal. Task identification can therefore be viewed as semantic clustering under a temporal-contiguity constraint. A Task is represented as

$$Task = \langle T, G, E, S \rangle,$$

where T denotes the task type, G the task goal, E the set of repository entities involved in the task, and S the Trace sequence assigned to the task.

Based on our preliminary study and informed by prior research on software-change intents and developer activities [22, 35, 45], we classify Tasks into six types:

$$T = \{\texttt{feature}, \texttt{bug fix}, \texttt{refactor}, \texttt{search}, \texttt{doc}, \texttt{mix}\}.$$

Agent A performs task identification. For long Trace nodes, rule-based compression $\hat{S} = H(S)$ retains the head and tail, capturing the task's origin and outcome, and replaces intermediate content with placeholders. Agent A can read the original Trace on demand through tool R to recover omitted details. It determines task boundaries from semantic relationships among Trace and identifies each task's type C_i and goal G_i. A regular-expression-based engine P then extracts repository entities E_i, such as files, from each Task cluster as code anchors for subsequent Rationale extraction and validation.

(2) Candidate Rationale Identification

After Task-level clustering, DevHound identifies Rationale from the Trace of each task. To ensure trustworthiness, the system does not represent a Rationale as an isolated piece of text. Instead, it adopts a semi-structured data model that jointly captures knowledge, metadata, and process evidence. A Rationale is defined as

$$\mathit{Rationale} = \langle K, y, p, E \rangle,$$

where K denotes a piece of tacit development knowledge distilled from the Task and tied to its repository entities; y denotes its knowledge type; $p \in [0, 1]$ is the confidence score of the candidate Rationale; and E is the evidence set supporting the knowledge, including repository entities and development Trace.

Drawing on existing research on design rationale and tacit knowledge [16, 24, 27, 31, 50, 52], we organize tacit development knowledge into five complementary Rationale types according to their primary informational roles, forming the type set $\mathcal{Y}$:

- **`intent`.** Explains why a code fragment or modification was designed in a particular way, including its historical context, trade-offs, and compatibility considerations.
- **`contract`.** Captures behavioral rules or conventions between code elements or modules that must be respected even though they are not explicitly declared in an interface.
- **`failure`.** Records potential errors, invalid outcomes, or latent failures caused by an implementation or usage pattern, together with their root causes.
- **`dependency`.** Describes implicit coupling or dependencies between a file and other modules, tools, environments, or external systems.
- **`solution`.** Summarizes reusable debugging, validation, or repair procedures established during development.

Given Task T_i, agent A uses its task type as a prior over the number and knowledge types of Rationale: `doc` and `search` may yield none, whereas `feature` and `bug fix` typically yield more candidates.

Guided by this prior knowledge, agent A identifies candidate Rationale supported by explicit developer statements, explicitly adopted suggestions, or subsequent implementation, debugging, or validation activities. It excludes restatements of code functionality, unadopted AI suggestions, content receiving no developer response, and descriptions lacking design intent, implicit contracts, risks, external dependencies, or reusable workflows. Each candidate must include its source Trace and corresponding repository entities, making its supporting evidence and the code it explains explicit.

(3) Quality Validation and Publication to Rationale Memory

Rationale candidates are not written directly to Memory; they must first undergo strict quality validation.

a. Process-Evidence Validation. For a candidate Rationale identified from Task T_i, its evidence set E must satisfy $E \subseteq S_i$: every evidence item must originate from the current Task, and its corresponding Trace must be locatable in the Task's original Trace sequence. In addition, the candidate's confidence score must meet the threshold $p \geq \theta = 0.8$.

b. Repository-Entity Validation. The entity set U referenced by a candidate Rationale must not only exist in the repository but also fall within the entity scope W of the current Task.

c. Semantic-Consistency Validation. The system compares a candidate Rationale with existing records in Rationale Memory M_R to determine whether their relationship is one of duplication, complementation, update, or conflict. Based on this comparison, the semantic validator V_s assigns the candidate an operation: $V_s(r, M_R) \in \{\texttt{insert}, \texttt{merge}, \texttt{update}, \texttt{reject}\}$. Here, `insert` adds a new Rationale, `merge` combines it with an existing record, `update` revises an existing record using new process evidence, and `reject` discards the candidate because it conflicts with existing evidence or lacks sufficient support.

Only Rationale candidates that pass all three validations enter Rationale Memory. The system then updates Rationale Memory according to the operation selected by semantic validation: $M_R \leftarrow \text{Update}(M_R, R_i)$.

3.2.3 ***Cognition Memory.*** A developer's familiarity varies across knowledge points. For example, a developer may be familiar with the programming language used by a project but unfamiliar with a particular framework or module. Based on this observation, DevHound does not assign a fixed, global ability label to a developer. Instead, it records cognitive evidence at the developer–knowledge-point level. Prior work has found that the frequency and recency of a developer's interactions with code entities can, to some extent, reflect their knowledge of the corresponding code [11]. This idea is also consistent with knowledge-tracing research, which dynamically estimates an individual's state by continuously observing behavior associated with a specific knowledge point rather than treating that state as a static attribute [8, 36].

Based on this idea, DevHound first identifies knowledge points from a Task and its Trace. A knowledge point is the smallest semantic unit that can be understood and explained independently, such as a programming language, framework, library, protocol, build tool, design pattern, project module, interface contract, error type, or software-engineering concept. For Task T_j, an LLM-based knowledge-point identification agent G analyzes its Trace and identifies the corresponding set of knowledge points, $\mathcal{K}_j = G(T_j) = \{k_1, k_2, \ldots, k_m\}$. DevHound then aggregates, within the corresponding Trace sequence, behavioral evidence relating developer u to knowledge point k_i and organizes it into a six-dimensional feature vector:

$$\mathbf{z}_{u,i} = \left(z^{\text{see}}, z^{\text{edit}}, z^{\text{run}}, z^{\text{ask}}, z^{\text{fail}}, z^{\text{succ}}\right)_{u,i}.$$

The six components correspond to exposure, editing, execution, inquiry, failure, and success behaviors. These features capture different aspects of the relationship between a developer and a knowledge point. Exposure records whether the developer has encountered the knowledge point; editing and execution indicate whether the developer has used it in practice; and inquiry, failure, and success capture performance while understanding or applying it. Distinguishing these behaviors is necessary because encountering a knowledge point does not imply being able to use it, and a single successful operation does not establish stable mastery. Each component is a time-decayed

count of the corresponding behavior:

$$z^a_{u,i} = \sum_{e \in \mathcal{B}^a_{u,i}} \exp\left(-\lambda \Delta t_e\right), \quad a \in \{\text{see, edit, run, ask, fail, succ}\}.$$

Here, $\mathcal{B}^a_{u,i}$ is the set of behaviors of type a, Δt_e is the time elapsed since behavior e, and λ controls temporal decay. Consequently, recent behaviors contribute more than earlier ones.

DevHound directly computes the cognitive-state value $c_{u,i} \in [0, 1]$ from feature vector $\mathbf{z}_{u,i}$ to estimate the likelihood that the developer is familiar with knowledge point k_i:

$$c_{u,i} = \text{sigmoid}\left(\sum_a \beta_a z^a_{u,i}\right).$$

Here, a ranges over the six behavior types above, and β_a is the weight of the corresponding component: editing, execution, and success have positive weights; inquiry and failure have negative weights; and mere exposure has a smaller positive weight. The time-decayed count of each behavior is multiplied by its weight, the products are summed, and sigmoid maps the sum to $[0, 1]$. A higher value indicates stronger behavioral evidence that the developer has mastered the knowledge point. In the absence of relevant behavior, $\mathbf{z}_{u,i} = \mathbf{0}$ and thus $c_{u,i} = 0.5$, avoiding an interpretation of missing evidence as either familiarity or unfamiliarity.

Finally, the record for developer u and knowledge point k_i in Cognition Memory is represented as $M_C(u, k_i) = \langle \mathbf{z}_{u,i}, c_{u,i}, t \rangle$, where t denotes the most recent update time. This state value is an evidence-based estimate used only to adjust explanation depth, rather than a direct measurement of the developer's actual cognitive ability; it does not alter the facts, evidence, or core conclusions in the answer.

4 Experimental Design

Evaluating DevHound requires a multi-stage, human-in-the-loop study because its effects span repository-level answer correctness, developer-specific cognitive needs, and Rationale Memory construction from real development processes. As shown in Figure 3, we recruit developers, observe their work in situ for eight weeks, and de-identify the data. We then construct datasets for the three RQs through expert annotation and participant labeling, and evaluate DevHound, baselines, and ablations using automated metrics and human assessment.

Our evaluation aims to answer the following three research questions:

- **RQ1. Effectiveness and Efficiency:** Can DevHound improve the correctness of Repository Q&A, and at what cost?
- **RQ2. Cognitive Needs:** Can DevHound better meet developers' cognitive needs while maintaining factual correctness?
- **RQ3. Memory Construction:** How effective is DevHound at constructing Rationale Memory from development processes, and what is the associated cost?

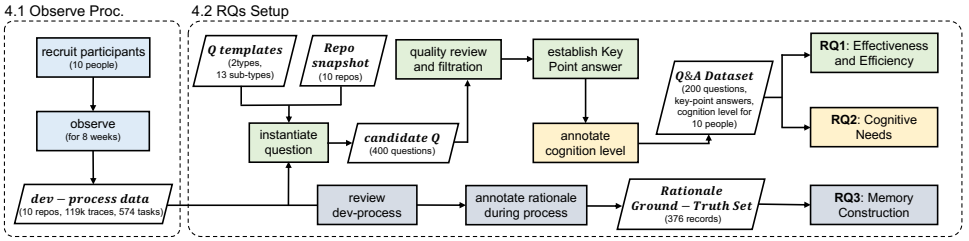


Fig. 3. Overview of the Experimental Procedure

4.1 Participant Recruitment & Overall Observation Procedure

Participants. We recruited 10 developers with diverse technical backgrounds following a set of strict eligibility criteria. Participants were required to satisfy three conditions: (1) having at least one year of software development experience; (2) being able to independently navigate and modify codebases larger than 20K LOC; and (3) having ongoing real-world development tasks during the observation period. To cover a diverse range of development scenarios, the recruitment process also considered participants' employment status, years of experience, primary technology stacks, and project domains. Table 1 summarizes their backgrounds.

Table 1. Backgrounds of Participants.

ID	Role	Exp. (yr)	Stack
P1	Ind. Dev.	7	Java, Go
P2	Ind. Dev.	5	TS, Vue
P3	Ph.D	4	Java, TS
P4	Ph.D	6	Rust, TS
P5	Ph.D	3	Python, C++
P6	Mast.	5	Python, Cython
P7	Mast.	3	Java, TS
P8	Mast.	2	Go, Python
P9	Bach.	3	Rust, TS
P10	Bach.	2	TS, Vue

Table 2. Repositories Included in the Study.

Repository	Lang.	Domain	Part.	LOC.
BitFun	Rust, TS	Desktop AI/Dev. Tools	P4	1,538k+
Continue	TS	AI Coding Assistant	P5	601k+
Gin	Go	Web Backend	P8	21k+
LangChain4j	Java	LLM Applications	P7	489k+
quju	Java, TS, Vue	Social Event Platform	P1, P2	23k+
React	JS	Web / Native UI	P3	819k+
AgentMeter	Rust, TS	AI Coding Analytics	P1, P2	26k+
scikit-learn	Python	Machine Learning	P6	297k+
Tauri	Rust, TS	Cross-Platform Apps	P9	33k+
virtualme2	TS, Vue	Dev. Analytics / IDE	P4, P10	122k+

Observation Process. At the beginning of Week 1, researchers helped participants install DevHound, register authorized local workspaces, and verify end-to-end data collection. During Weeks 1–8, participants worked on their ongoing development tasks in their repositories while DevHound unobtrusively collected data from these workspaces. After Week 8, we gathered the recorded process data and removed identifying and privacy-sensitive information. Across 10 participants, the observation yielded 1,736 hours of development activity, 118,742 Traces, and 574 Tasks. This corpus captures real-world development processes and supports evaluation dataset construction.

4.2 RQs Setup

4.2.1 ***RQ1. Effectiveness and Efficiency:*** *Can DevHound improve the correctness of Repository Q&A, and at what cost?*

Q&A Dataset. To cover diverse question types and difficulty levels in realistic repository scenarios, we adopt question-template instantiation, a commonly used strategy for constructing code QA datasets [5, 25, 39]. **(1) Question Instantiation.** We instantiate templates representing common developer intents and question types with concrete repository entities and extend the resulting questions, producing 400 candidates. Crucially, a key purpose of the eight-week observation is to retrospectively construct Process-based Questions targeting only actual development decisions captured in Trace. Their answers require the corresponding process evidence. After deduplication and manual quality review, we select 10 Snapshot-based and 10 Process-based Questions per repository, yielding 200 questions (Table 3). **(2) Key Point-based Answer Establishment.** Following prior Repository Q&A work, we use a 10-point rubric and decompose each reference answer into n independently verifiable **key points**, each specifying an atomic **fact**, its **score**, and supporting **evidence**, such as repository references and Trace snippets. Human experts draft the reference answers and key points with assistance from Claude-Fable-5.1 and Claude Code, then review and revise them to ensure correctness.

Table 3. Distribution and Examples of RQ1 Question Types.

Type	Subtype	N	Example Question
Snapshot-based	S1 Architecture	13	Which package supplies the sidebar Webview, and what does its development script run?
	S2 Entities & Roles	15	For an unsupported city, which city's configuration, zoom level, and attraction district are returned?
	S3 Location	15	Which method exposes Gin's BSON binder type to the generic binding interface?
	S4 Dependencies	14	When does CodePath emit a segment at joins or loops, and can it emit the same segment twice?
	S5 Behavior & Mechanism	17	How does the local knowledge tree decide whether a file still needs knowledge analysis?
	S6 Usage & Configuration	13	Which changelog headings cover build/CI and documentation, and do they use the same ordering value?
	S7 Runtime & Diagnostics	13	How does `isAbortError` classify strings, errors, and objects, and when do `DOMException` tests run?
Process-based	H1 Decision Drivers	16	Why were `contenteditable` elements excluded from high-DPI forced compositing?
	H2 Selection Rationale	19	Why did `RepoKnowledgeTool` read repository JSONL directly instead of invoking an external CLI?
	H3 Alternatives/Trade-offs	18	Why use `POST /teams/{id}/leave` instead of `DELETE /teams/{id}/members/me` for self-removal?
	H4 Parameters & Strategy	16	Why are candidate tasks hard-split by file, day, a 6-hour gap, or a 20-turn limit?
	H5 Evolution & Change	15	Why require explicit decisions or implementation evidence during knowledge extraction?
	H6 Validation Outcomes	16	Why did active-draft submission still fail after `validateForSubmit` was changed?
	Total	200	

Baselines. We use five external baselines: three general-purpose coding agents, **Cursor** [4], **Codex** [32], and **Claude Code** [3]; and two open-source repository-level Q&A approaches, **SWE-QA-Agent** [38] and **DeepRepoQA** [37].

Ablation Study. **w/o QUE** removes the question understanding and enhancement module, directly feeding the original question into the planning–retrieval–verification loop; **w/o RM** removes the Rationale Memory and its retrieval tools, while keeping the remaining model, prompts, repository snapshot retrieval tools, and solving budget identical to those in the Full setting; **w/o CM** removes the Cognition Memory and uniformly organizes the content and generates answers at the regular level.

Metrics.

(1) Key Point-based Answer Correctness (KAC). Following prior work, we measure answer correctness using Key Point-based Answer Correctness (KAC). For a candidate answer A, its score is

$$\mathrm{KAC}(A) = \sum_{i \in M(A)} s_i,$$

where $M(A)$ is the set of scoring points correctly covered by A, and s_i is the score assigned to point i. Because the scoring points for each question sum to 10, $\mathrm{KAC}(A)$ ranges from 0 to 10. We use Claude-Fable-5.1 and GPT-6-Astra as judges, randomize candidate-answer order, and manually verify all judgments.

(2) Times & Tokens. We record the end-to-end latency from question submission to the generation of the final answer, as well as the total number of input and output tokens consumed during the process. Tokens incurred by tool calls are also included in the total token count.

4.2.2 ***RQ2. Cognitive Needs:*** *Can DevHound better meet developers' cognitive needs while maintaining factual correctness?*

Cognitive Needs Level Annotation. For each of the 200 Q&A pairs constructed in Section 4.2.1, we asked all 10 participants to review the key points of its reference answer and annotate their own cognitive needs for each key point. Specifically, for each key point, participants labeled their cognitive need according to their current level of understanding and desired depth of explanation as

Novice, **Regular** or **Expert**. As a result, each question–key point pair received 10 participant-level annotations. These participant-specific labels served as ground truth for evaluating DevHound's Cognitive Need predictions.

Baseline. We select **Claude Code (Sonnet 5)**, which achieved the best answer correctness in RQ1, as the baseline for RQ2.

Metrics.

- **Answer-level Cognitive Fit.** We conduct a human evaluation using a five-point Likert scale (1 = not satisfied, 5 = fully satisfied). For each participant–question pair, participants independently rate the anonymized answers from DevHound (Full) and Claude Code in randomized order along five dimensions: **(1) Explanation Sufficiency**, whether sufficient background, reasoning, and details are provided; **(2) Non-redundancy**, whether unnecessary explanations are avoided; **(3) Clarity**, whether the answer is well organized and easy to follow; **(4) Cognitive Relevance**, whether it focuses on the knowledge currently needed; and **(5) Task Usefulness**, whether it helps resolve the question or determine the next action.
- **Cognitive-level Prediction Accuracy.** Both DevHound predictions and participant annotations use three levels: **Novice**, **Regular**, and **Expert**. We establish one-to-one semantic matches between generated and reference key points, independent of their level labels. Let G be all reference key points, $M \subseteq G$ the matched points, and p_i and h_i the predicted and annotated levels of point i. **(1) Accuracy (Acc)** measures the proportion of matched key points assigned the correct cognitive level: $\mathbf{Acc} = \frac{\sum_{i \in M} \mathbf{1}(p_i = h_i)}{|M|}$. **(2) Macro-F1** measures class-balanced prediction performance over the matched key points by averaging the F1 scores of the three levels: $\mathbf{Macro\text{-}F1} = \frac{1}{3} \sum_c \frac{2P_c R_c}{P_c + R_c}$, where P_c and R_c are the precision and recall of level c. **(3) End-to-end Cognitive Alignment (ECA)** measures the proportion of all reference key points that are both covered and assigned the correct cognitive level: $\mathbf{ECA} = \frac{\sum_{i \in M} \mathbf{1}(p_i = h_i)}{|G|}$. Here, $\mathbf{1}(\cdot)$ equals 1 when the two levels match and 0 otherwise.

4.2.3 ***RQ3. Memory Construction:*** *How effective is DevHound at constructing Rationale Memory from development processes, and what is the associated cost?*

Evaluation Object and Gold Set. RQ3 evaluates the construction of Rationale Memory from development Traces and repository snapshots, rather than the answers generated in RQ1 and RQ2, so its evaluation objects are the resulting Rationale records.

For each repository, annotators who had participated in the observed development tasks or were familiar with the repository independently reviewed its snapshot and complete observed Trace. The annotators identified all evidence-supported Rationale records that occurred during the observation period, and disagreements were resolved by a third annotator. This process produced a gold set of 376 Rationale records.

Models. We select six models to cover a broad range of price, scale, deployment form, and reasoning capability. The resulting coverage is as follows:

- **High-cost frontier:** GPT-5.6 Sol [33], representing the strongest general reasoning and code-analysis tier in our model set.
- **Medium-cost, high-capability:** GPT-5.6 Terra [34] and Claude Sonnet 5 [2], representing general-purpose models with strong reasoning, coding, and tool-use capabilities.
- **Low-cost, efficiency-oriented:** DeepSeek V4-1 Flash [9] and GLM-5.3 Flash [48], representing models optimized for lower-latency and lower-cost inference.

- **Ultra-low-cost, compact:** Llama-3.1-8B-Instruct [30], providing a substantially smaller open-weight baseline.

Ablation Study. We compare the complete method with two single-component ablations:

- **w/o Validation** Bypasses all three validation steps, publishing every identified candidate.
- **w/o Task** Replaces semantic Task construction with fixed-length sliding windows over the Trace sequence. To keep validation comparable, the source scope is the Task in the full configuration and the corresponding Trace window in this ablation.

Metrics. Let $\mathcal{S}$ and $\mathcal{G}$ denote the generated and gold Rationale sets. Because each Rationale record expresses one atomic piece of development knowledge, matching is based on semantic equivalence. Let $\kappa(r)$ denote the atomic knowledge expressed by record r. For $s \in \mathcal{S}$ and $g \in \mathcal{G}$, we define

$$m(s, g) = \begin{cases} 1, & \kappa(s) \equiv \kappa(g), \\ 0, & \text{otherwise}, \end{cases}$$

where $\kappa(s) \equiv \kappa(g)$ means that the two records concern the same knowledge subject and convey the same core claim. Two blinded reviewers independently establish the matches, and a third reviewer resolves disagreements. The final match set $\mathcal{M}$ is one-to-one: each generated or gold record can appear in at most one pair.

The matching $\mathcal{M}$ yields three exhaustive construction outcomes:

- **Correct:** matched generated records, $N_C = |\mathcal{M}|$.
- **Spurious:** unmatched generated records, $N_S = |\mathcal{S}| - |\mathcal{M}|$.
- **Missed:** unmatched gold records, $N_M = |\mathcal{G}| - |\mathcal{M}|$.

We then compute

$$P = \frac{N_C}{N_C + N_S}, \qquad R = \frac{N_C}{N_C + N_M}, \qquad F1 = \frac{2PR}{P + R}.$$

We report the three outcome counts together with Precision, Recall, and F1. Construction cost is the total model-invocation cost required to process the complete eight-week corpus, including all pipeline stages and retries. Prices are normalized to US dollars using the provider prices recorded at the time of the experiment; all configurations for a model process the same corpus, making their total costs directly comparable.

5 Results

5.1 RQ1. Effectiveness and Efficiency: Can DevHound improve the correctness of Repository Q&A, and at what cost?

Table 4 reports the answer correctness and solving cost of DevHound, the external baselines, and the three ablated variants.

Overall Effectiveness. DevHound achieves the highest overall KAC across all four foundation models, ranging from 8.45 to 9.00. Under the same model, it outperforms Cursor, SWE-QA-Agent, and DeepRepoQA by 2.34–2.79 points, and raises the KAC of standalone Claude Code and Codex from 6.72 and 6.62 to 9.00 and 8.85, respectively. These consistent gains show that the improvement is not specific to one foundation model.

Where the Gains Come From. The improvement is concentrated on Process-based Questions, where DevHound achieves 8.53–9.03 KAC compared with 3.72–4.72 for the baselines. On Snapshot-based Questions, it achieves 8.36–8.97 and remains comparable to or moderately better than the baselines. DevHound thus closes the large gap between the two question types by recovering development-process evidence unavailable from the current repository snapshot.

Table 4. Answer correctness (on a 10-point scale) and solving efficiency across models and tools.

Model	Tool	KAC			Time (s)			Tokens (k)		
		Snap.	Proc.	All	Snap.	Proc.	All	Snap.	Proc.	All
Claude Code(Sonnet 5)		8.72	4.72	6.72	118	221	169.5	398.3	552.8	475.6
Codex(GPT 5.6 Terra)		8.68	4.55	6.62	120	238	179	411.9	566.8	489.4
Sonnet 5	DevHound	**8.97**	**9.03**	**9.00**	140	164	152	458.4	523.9	491.2
	w/o QUE	8.67	8.63	8.65 (-0.35)	132	**153**	**142.5** (-9.5)	438.9	**501.5**	**470.2** (-21)
	w/o RM	8.65	5.02	6.84 (-2.16)	**126**	218	172 (+20)	**420.5**	560.7	490.6 (-1)
	w/o CM	8.95	9.01	8.98 (-0.02)	137	160	148.5 (-3.5)	451.7	515.9	483.8 (-7)
	Cursor	8.46	4.46	6.46 (-2.54)	127	257	192 (+40)	423.4	604.0	513.7 (+23)
	SWE-QA-Agent	8.31	4.18	6.25 (-2.75)	148	239	193.5 (+41.5)	470.6	582.4	526.5 (+35)
	DeepRepoQA	8.55	4.62	6.59 (-2.41)	163	281	222 (+70)	516.4	671.9	594.1 (+103)
GPT 5.6 Terra	DevHound	8.74	**8.95**	**8.85**	131	166	148.5	446.4	530.0	488.2
	w/o QUE	8.44	8.55	8.50 (-0.35)	**123**	**155**	**139** (-9.5)	**427.5**	**507.1**	**467.3** (-21)
	w/o RM	**8.76**	4.72	6.74 (-2.11)	126	224	175 (+26.5)	430.6	575.8	503.2 (+15)
	w/o CM	8.72	8.93	8.83 (-0.02)	128	162	145 (-3.5)	439.9	521.4	480.6 (-8)
	Cursor	8.74	4.28	6.51 (-2.34)	129	270	199.5 (+51)	443.8	594.0	518.9 (+31)
	SWE-QA-Agent	8.18	3.94	6.06 (-2.79)	143	246	194.5 (+46)	461.9	590.8	526.3 (+38)
	DeepRepoQA	8.37	4.40	6.39 (-2.46)	157	289	223 (+74.5)	503.7	684.2	594.0 (+106)
DeepSeek V4.1 Flash	DevHound	**8.36**	**8.53**	**8.45**	104	138	121	418.8	498.9	458.9
	w/o QUE	8.02	8.12	8.07 (-0.38)	**97**	**128**	**112.5** (-8.5)	399.4	**477.7**	**438.6** (-20)
	w/o RM	8.18	4.10	6.14 (-2.31)	101	211	156 (+35)	407.8	551.4	479.6 (+21)
	w/o CM	8.34	8.50	8.42 (-0.03)	101	134	117.5 (-3.5)	412.5	491.2	451.9 (-7)
	Cursor	8.12	3.98	6.05 (-2.40)	105	226	165.5 (+44.5)	**398.6**	558.7	478.7 (+20)
	SWE-QA-Agent	7.75	3.72	5.74 (-2.71)	116	209	162.5 (+41.5)	429.5	550.4	490.0 (+31)
	DeepRepoQA	7.98	4.05	6.02 (-2.43)	128	243	185.5 (+64.5)	468.5	625.4	547.0 (+88)
GLM 5.3 Flash	DevHound	**8.52**	**8.68**	**8.60**	109	143	126	426.2	506.3	466.3
	w/o QUE	8.17	8.28	8.23 (-0.37)	**102**	**133**	**117.5** (-8.5)	407.1	**485.5**	**446.3** (-20)
	w/o RM	8.24	4.19	6.22 (-2.38)	106	218	162 (+36)	414.2	560.6	487.4 (+21)
	w/o CM	8.49	8.66	8.58 (-0.02)	106	139	122.5 (-3.5)	420.1	498.6	459.4 (-7)
	Cursor	8.28	4.05	6.17 (-2.43)	110	232	171 (+45)	**406.0**	569.9	487.9 (+22)
	SWE-QA-Agent	7.91	3.81	5.86 (-2.74)	121	215	168 (+42)	438.3	561.0	499.6 (+33)
	DeepRepoQA	8.11	4.16	6.14 (-2.46)	134	250	192 (+66)	477.9	637.3	557.6 (+91)

Note: Green indicates better performance, and red indicates worse performance. Higher KAC is better; lower time and token consumption are better.

Component Contributions. Removing Rationale Memory decreases overall KAC by 2.11–2.38 points and Process-based KAC by 4.01–4.49 points, while changing Snapshot-based KAC by at most 0.32 points. This confirms that Rationale Memory supplies the historical decision evidence required by Process-based Questions. Removing question understanding lowers overall KAC by a smaller but consistent 0.35–0.38 points. Removing Cognition Memory changes KAC by only 0.02–0.03 points, consistent with its role in adapting explanation focus and depth rather than supplying factual evidence; this contribution is evaluated in RQ2.

Efficiency. DevHound requires 121–152 seconds and 458.9–491.2k tokens per question, making it 40–74.5 seconds faster and 20–106k tokens cheaper than the same-model repository-level QA baselines. Removing Rationale Memory increases Process-based latency by 54–75 seconds, suggesting that direct access to historical evidence avoids prolonged repository-only exploration. Question understanding adds only 8.5–9.5 seconds and 20–21k tokens, while cognition adaptation adds 3.5 seconds and 7–8k tokens. The correctness gains therefore do not result from a larger solving budget.

Answer to RQ1. *DevHound raises overall KAC to 8.45–9.00 across foundation models, primarily by using Rationale Memory to improve Process-based Questions by 4.01–4.49 points. It achieves these gains with lower overall latency and token consumption than same-model repository-level QA baselines, while the other modules introduce only modest overhead.*

5.2 RQ2. Cognitive Needs: Can DevHound better meet developers' cognitive needs while maintaining factual correctness?

Fig. 4. Answer-level Cognitive Fit.

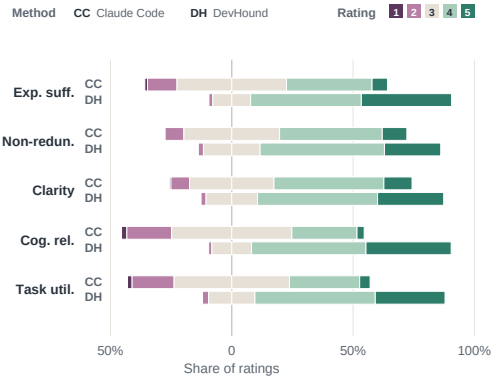


Table 5. Cognitive-level Prediction Accuracy.

Metric	Result
Acc	83.0%
Macro-F1	83.4%
ECA	78.1%

Table 6. Confusion Matrix for Matched Key Points.

Human Label	**Novice**	**Regular**	**Expert**	*N*
Novice	1,594 (83.0%)	300 (15.6%)	26 (1.4%)	1,920
Regular	230 (9.0%)	2,099 (82.0%)	231 (9.0%)	2,560
Expert	26 (1.4%)	275 (14.3%)	1,619 (84.3%)	1,920

Answer-level Cognitive Fit. DevHound's rating distributions consistently shift toward scores of 4–5 across all five dimensions. The improvements are most pronounced in Cog. Rel., Task Util., and Exp. Suff., with average score increases of 1.06, 0.88, and 0.85, respectively. These results suggest that DevHound better explains what developers currently need to understand and more effectively supports them in completing their ongoing tasks. Meanwhile, Non-redun. and Clarity also improve by 0.41 and 0.42, respectively, indicating that these gains do not simply result from longer answers. Instead, DevHound better calibrates the focus and depth of its explanations, providing necessary information while reducing redundancy and improving content organization, thereby enhancing the overall fit between its answers and developers' cognitive needs.

Cognitive-level Prediction Accuracy. As shown in Table 5, DevHound achieves 83.0% Accuracy and 83.4% Macro-F1 on matched key points, demonstrating accurate and balanced predictions across the three cognitive levels. Table 6 shows that most errors occur between adjacent levels, while Novice–Expert misclassifications account for only 1.4%. When unmatched key points are also considered, the ECA remains 78.1%, indicating reliable end-to-end explanation-depth adaptation.

Answer to RQ2. *DevHound better meets developers' cognitive needs, improving all five answer-level cognitive-fit dimensions while achieving 83.0% cognitive-level prediction accuracy and 78.1% end-to-end cognitive alignment.*

5.3 RQ3. Memory Construction: How effective is DevHound at constructing Rationale Memory from development processes, and what is the associated cost?

Table 7 reports the effectiveness and construction cost of DevHound and its two ablated variants across the six foundation models.

Table 7. Effectiveness and cost of constructing Rationale Memory (C: Correct; S: Spurious; M: Missed).

Method	Model	C ↑	S ↓	M ↓	P ↑	R ↑	F1 ↑	Cost ↓
DevHound	gpt-5.6-sol	362	20	14	0.948	0.963	0.955	$46.79
	gpt-5.6-terra	364	15	12	0.960	0.968	0.964	$23.78
	claude-sonnet-5	352	29	24	0.924	0.936	0.930	$23.39
	deepseek-v4-1-flash	342	42	34	0.891	0.910	0.900	$3.45
	glm-5.3-flash	350	24	26	0.936	0.931	0.933	$1.36
	llama-3.1-8b-instruct	292	89	84	0.766	0.777	0.771	$0.31
w/o Validation	gpt-5.6-sol	361	95	15	0.792	0.960	0.868	$45.14
	gpt-5.6-terra	362	106	14	0.774	0.963	0.858	$22.96
	claude-sonnet-5	353	96	23	0.786	0.939	0.856	$22.56
	deepseek-v4-1-flash	335	147	41	0.695	0.891	0.781	$3.33
	glm-5.3-flash	330	141	46	0.701	0.878	0.779	$1.31
	llama-3.1-8b-instruct	270	168	106	0.616	0.718	0.663	$0.29
w/o Task	gpt-5.6-sol	300	44	76	0.872	0.798	0.833	$29.70
	gpt-5.6-terra	288	49	88	0.855	0.766	0.808	$15.10
	claude-sonnet-5	294	52	82	0.850	0.782	0.814	$14.81
	deepseek-v4-1-flash	274	68	102	0.801	0.729	0.763	$2.23
	glm-5.3-flash	267	65	109	0.804	0.710	0.754	$0.89
	llama-3.1-8b-instruct	200	110	176	0.645	0.532	0.583	$0.21

Overall Construction Effectiveness. Across the six foundation models, DevHound correctly constructs 292–364 of the 376 gold Rationale records and achieves F1 scores of 0.771–0.964; five models reach at least 0.900 F1. These results show that DevHound reliably constructs Rationale Memory across model families.

Capability–Cost Trade-off. DeepSeek V4-1 Flash and GLM-5.3 Flash achieve 0.900 and 0.933 F1 at only $3.45 and $1.36, while GPT-5.6 Terra reaches 0.964 at $23.78; the compact 8B model drops to 0.771. Thus, low-cost, efficiency-oriented models can learn effectively from development processes and construct high-quality Rationale Memory, while stronger models offer limited gains once sufficient capability is reached.

Contributions of Pipeline Stages. Both Task construction and Validation improve all six models, with larger average F1 gains for the three lower-capability models than for the three higher-capability models (Task: 0.168 vs. 0.131; Validation: 0.127 vs. 0.089). Task construction reduces both Missed and Spurious Rationales because semantic boundaries avoid mixing different goals or splitting related evidence, whereas Validation mainly reduces Spurious Rationales by filtering unsupported or inconsistent candidates.

> ***Answer to RQ3.*** *DevHound reliably constructs Rationale Memory across models. Low-cost, efficiency-oriented models perform well once they reach sufficient capability, beyond which stronger models provide limited gains. Task construction and Validation both improve construction quality, particularly for lower-capability models.*

6 Discussion

6.1 Limits of Process-based Q&A

DevHound's ability to answer Process-based Questions depends on how explicitly decision grounds are recorded and how concentrated the relevant evidence is. Explicit decision statements and complete evidence within a single Task provide direct support. Recovering alternatives, changes across development stages, or relationships between modifications and validation requires more complete

process records and the preservation of evidence relationships in Memory. Retaining final decisions alone is therefore insufficient; the context in which those decisions emerged and changed must also be preserved.

This boundary also means that answer failures cannot simply be attributed to model reasoning. Necessary evidence may never have been recorded, may have been omitted during Memory construction, may not have been retrieved, or may have been incorrectly linked or overinterpreted during generation. Improving Process-based Q&A thus requires better evidence coverage, knowledge retention, and evidence use. When historical grounds are missing, even a plausible explanation cannot substitute for the actual decision reason.

6.2 Generalizability

DevHound's approach can be extended to development tasks such as code modification, bug fixing, and code review. As task goals change, the knowledge to be learned, the specific contents of Memory, and the organization of outputs can be adapted, while the core methodology remains unchanged: learning from development processes, linking knowledge to its source evidence, and validating its quality to ensure trustworthiness. For example, bug fixing emphasizes failure causes and validation procedures, whereas code review emphasizes design constraints and historical tradeoffs. What generalizes is the method for constructing and using trustworthy knowledge from developer behavior; specific knowledge still requires validation against the target task and current repository.

6.3 Threats to Validity

Construct Validity. Answer correctness, cognitive-fit ratings, and Rationale extraction quality measure different aspects of the system, but do not fully represent long-term understanding or actual development performance. Developer behavior is also only indirect evidence of cognitive state and cannot be equated with actual mastery.

Internal Validity. Annotators' repository familiarity, model-assisted evaluation, and runtime stochasticity may affect the results. Manual verification, anonymized evaluation, and randomized answer order mitigate some biases. However, external baseline comparisons primarily support system-level effects; component contributions require further assessment through controlled ablations.

External Validity. The results are limited by the participants, repositories, tool ecosystems, and observation period. Process-based Questions are constructed around recorded development decisions, and missing evidence in practical use may reduce effectiveness. Applications beyond Q&A and cross-repository generalization require further validation.

Conclusion Validity. Observations associated with the same participant, question, or repository are dependent and cannot all be treated as independent samples. Sample size, inter-rater agreement, and multiple comparisons may also affect conclusions, requiring appropriate statistical analysis and uncertainty estimates when interpreting the results.

7 Related Work

Repository-Level Q&A and Coding Agents. Early Q&A benchmarks, such as CodeQA and InfiBench, focus on individual functions, code snippets, or self-contained programming problems [23, 25]. RepoChat, SWE-QA, SWE-QA-Pro, and CodeAssistBench extend Q&A to repository-level, cross-file, multi-hop, and multi-turn understanding [1, 7, 19, 39], while coding agents such as SWE-agent and OpenHands explore repositories through planning, retrieval, and tool use [44, 46].

However, their evidence remains primarily repository snapshots and derived indices. Even when questions originate from GitHub issues, the issues are generally treated as question sources rather than development-process evidence traceably linked to the current implementation [19, 39]. These methods therefore reveal what the code does better than why a design was adopted, which alternatives were rejected, or how the final choice was validated. VirtualME personalizes assistance using a global developer profile [26], but does not model comprehension needs for individual knowledge points. DevHound instead combines the current repository with Rationale Memory to support current facts and historical decisions, and uses Cognition Memory only to adjust the explanation depth of each knowledge point without changing facts, evidence, or conclusions.

Knowledge Mining from Software Repositories and Development Processes. DRMiner and Automated Extraction and Analysis of Developer's Rationale extract solutions, arguments, and decision rationale from Jira discussions and commit messages, respectively [10, 50]. Such retrospective artifacts, however, may not preserve the complete path by which a decision was proposed, adopted, implemented, and validated. DevHound therefore captures IDE operations and human–agent interactions as development unfolds and distills Rationale from the continuous process. Lingxi and ExpeRepair instead distill procedural knowledge, repair cases, and abstract experience from historical issue–patch pairs and agent repair trajectories to guide later tasks [31, 47]. In short, these memories mainly preserve *how* to perform a class of tasks; Rationale Memory preserves *why* developers made a particular choice by linking the adopted solution and its reasons to process evidence and repository entities.

Personalization and Cognitive Support for Developers. Development behavior can indicate programmers' code familiarity or cognitive load: prior work uses interaction frequency and duration, physiological signals, or repeated gaze behavior to estimate knowledge and comprehensibility [11, 12, 14]. Recent work further adapts code explanations to developers' backgrounds, goals, profiles, expertise, roles, or styles [6, 26, 28, 42], consistent with the expertise reversal effect [17]. However, a global profile cannot represent a developer's familiarity with every knowledge point. DevHound therefore models cognition at the developer–knowledge-point level from point-related behavioral evidence and adjusts only the depth of each explanation, leaving its facts, evidence, and core conclusions unchanged.

8 Conclusion

This paper presented DevHound, a repository-level question-answering approach that continuously observes and learns from development processes to improve answer correctness and cognitive fit. It combines repository evidence with historical decision knowledge and supporting process evidence preserved in Rationale Memory, and uses Cognition Memory to adapt explanation depth to developers' familiarity with individual key points. An eight-week naturalistic study involving 10 developers and 10 repositories shows improved answer correctness across four foundation models, especially on Process-based Questions, with lower latency and token consumption than same-model repository-level QA baselines. DevHound also improves all five cognitive-fit dimensions and achieves 83.0% cognitive-level prediction accuracy on matched key points. Low-cost models can construct high-quality Rationale Memory, with Task construction and Validation both improving construction quality. Together, these findings highlight the value of development traces as a complementary knowledge source for repository-level Q&A. Learning from both decision evidence and cognitive evidence supports answers that explain how repository decisions arose and present those explanations at an appropriate depth. Its effectiveness depends on the coverage and quality of process evidence, and generalization to other development tasks requires further evaluation.

9 Data Availability

The artifacts of this paper are publicly available at https://devhound2027.github.io/DevHound/.

References

[1] Samuel Abedu, Laurine Menneron, SayedHassan Khatoonabadi, and Emad Shihab. 2025. RepoChat: An LLM-Powered Chatbot for GitHub Repository Question-Answering. In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE, Ottawa, ON, Canada, 255–259. doi:10.1109/MSR66628.2025.00045

[2] Anthropic. 2026. Introducing Claude Sonnet 5. Retrieved October 3, 2026 from https://www.anthropic.com/research/claude-sonnet-5

[3] Anthropic. n.d. Claude Code: Overview. Retrieved October 3, 2026 from https://code.claude.com/docs/en/overview

[4] Anysphere. n.d. Cursor Agent. Retrieved October 3, 2026 from https://cursor.com/docs/agent/overview

[5] Aakash Bansal, Zachary Eberhart, Lingfei Wu, and Collin McMillan. 2021. A Neural Question Answering System for Basic Questions about Subroutines. In *2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 60–71. doi:10.1109/SANER50967.2021.00015

[6] Michelle Brachman, Arielle Goldberg, Andrew Anderson, Stephanie Houde, Michael Muller, and Justin D. Weisz. 2025. Towards Personalized and Contextualized Code Explanations. In *Adjunct Proceedings of the 33rd ACM Conference on User Modeling, Adaptation and Personalization*. Association for Computing Machinery, New York, NY, USA, 120–125. doi:10.1145/3708319.3733681

[7] Songcheng Cai, Zhiheng Lyu, Yuansheng Ni, Xiangchao Chen, Baichuan Zhou, Shenzhe Zhu, Yi Lu, Haozhe Wang, Chi Ruan, Benjamin Schneider, Weixu Zhang, Xiang Li, Andy Zheng, Yuyu Zhang, Ping Nie, and Wenhu Chen. 2026. SWE-QA-Pro: A Representative Benchmark and Scalable Training Recipe for Repository-Level Code Understanding. In *Findings of the Association for Computational Linguistics: ACL 2026*. Association for Computational Linguistics, San Diego, California, United States, 16958–16999. doi:10.18653/v1/2026.findings-acl.837

[8] Albert T. Corbett and John R. Anderson. 1994. Knowledge Tracing: Modeling the Acquisition of Procedural Knowledge. *User Modeling and User-Adapted Interaction* 4, 4 (1994), 253–278. doi:10.1007/BF01099821

[9] DeepSeek. 2026. DeepSeek-V4.1-Flash: Pushing the Limits of KV Cache Compression. Retrieved October 3, 2026 from https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash

[10] Mouna Dhaouadi, Bentley Oakes, and Michalis Famelis. 2025. Automated Extraction and Analysis of Developer's Rationale in Open Source Software. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 2548–2570. doi:10.1145/3729383

[11] Thomas Fritz, Gail C. Murphy, and Emily Hill. 2007. Does a Programmer's Activity Indicate Knowledge of Code?. In *Proceedings of the 6th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering*. Association for Computing Machinery, New York, NY, USA, 341–350. doi:10.1145/1287624.1287673

[12] Hao Gao, Haytham Hijazi, Júlio Medeiros, João Durães, Chan Tong Lam, Paulo de Carvalho, and Henrique Madeira. 2025. NRevisit: A Cognitive Behavioral Metric for Code Understandability Assessment. In *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering*. Association for Computing Machinery, New York, NY, USA, 908–918. doi:10.1145/3756681.3757053

[13] Jie Gao, Yue Xue, Xiaofei Xie, Junming Cao, SoeMin Thant, Erika Lee, and Bowen Xu. 2026. Understanding Codebase like a Professional! Human–AI Collaboration for Code Comprehension. In *Proceedings of the 2026 34th IEEE/ACM International Conference on Program Comprehension*. Association for Computing Machinery, New York, NY, USA, 343–354. doi:10.1145/3794763.3794822

[14] Lucian José Gonçales, Kleinner Farias, and Bruno C. da Silva. 2021. Measuring the Cognitive Load of Software Developers: An Extended Systematic Mapping Study. *Information and Software Technology* 136 (2021), 106563. doi:10.1016/j.infsof.2021.106563

[15] Mia Mohammad Imran and Kostadin Damevski. 2022. Using Clarification Questions to Improve Software Developers' Web Search. *Information and Software Technology* 151 (2022), 107021. doi:10.1016/j.infsof.2022.107021

[16] Xue Jiang, Tianyu Zhang, Lingwei Wu, Ziyu Wang, Ge Li, Yuan Sui, Hao Zhu, Wenpin Jiao, Zhi Jin, and Yihong Dong. 2026. PRAXIS: Graph-Grounded Tacit Knowledge for Domain Code Generation. arXiv:2608.19784 [cs.SE] doi:10.48550/arXiv.2608.19784

[17] Slava Kalyuga, Paul Ayres, Paul Chandler, and John Sweller. 2003. The Expertise Reversal Effect. *Educational Psychologist* 38, 1 (2003), 23–31. doi:10.1207/S15326985EP3801_4

[18] Mik Kersten and Gail C. Murphy. 2006. Using Task Context to Improve Programmer Productivity. In *Proceedings of the 14th ACM SIGSOFT International Symposium on Foundations of Software Engineering*. Association for Computing Machinery, New York, NY, USA, 1–11. doi:10.1145/1181775.1181777

[19] Myeongsoo Kim, Shweta Garg, Baishakhi Ray, Varun Kumar, and Anoop Deoras. 2025. CodeAssistBench (CAB): Dataset and Benchmarking for Multi-turn Chat-Based Code Assistance. In *Advances in Neural Information Processing Systems*, Vol. 38. Curran Associates, Inc., San Diego, California, USA and Mexico City, Mexico, 142563–142597. doi:10.52202/085713-4280

[20] Amy J. Ko, Robert DeLine, and Gina Venolia. 2007. Information Needs in Collocated Software Development Teams. In *29th International Conference on Software Engineering (ICSE'07)*. IEEE Computer Society, Los Alamitos, CA, USA, 344–353. doi:10.1109/ICSE.2007.45

[21] Aayush Kumar, Yasharth Bajpai, Sumit Gulwani, Gustavo Soares, and Emerson Murphy-Hill. 2025. Why AI Agents Still Need You: Findings from Developer-Agent Collaborations in the Wild. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, Seoul, Republic of Korea, 432–444. doi:10.1109/ASE63991.2025.00043

[22] Timothy C. Lethbridge, Janice Singer, and Andrew Forward. 2003. How Software Engineers Use Documentation: The State of the Practice. *IEEE Software* 20, 6 (2003), 35–39. doi:10.1109/MS.2003.1241364

[23] Linyi Li, Shijie Geng, Zhenwen Li, Yibo He, Hao Yu, Ziyue Hua, Guanghan Ning, Siwei Wang, Tao Xie, and Hongxia Yang. 2024. InfiBench: Evaluating the Question-Answering Capabilities of Code Large Language Models. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., Vancouver, Canada, 128668–128698. doi:10.52202/079017-4087

[24] Zhenmin Li and Yuanyuan Zhou. 2005. PR-Miner: Automatically Extracting Implicit Programming Rules and Detecting Violations in Large Software Code. In *Proceedings of the 10th European Software Engineering Conference Held Jointly with the 13th ACM SIGSOFT International Symposium on Foundations of Software Engineering*. Association for Computing Machinery, New York, NY, USA, 306–315. doi:10.1145/1081706.1081755

[25] Chenxiao Liu and Xiaojun Wan. 2021. CodeQA: A Question Answering Dataset for Source Code Comprehension. In *Findings of the Association for Computational Linguistics: EMNLP 2021*. Association for Computational Linguistics, Punta Cana, Dominican Republic, 2618–2632. doi:10.18653/v1/2021.findings-emnlp.223

[26] Yuhong Liu, Yunhe Su, Zhipeng Peng, Zhiwen Luo, Lin Shi, Zhi Jin, and Li Zhang. 2026. On the Road to Personalized Code Intelligence: Portraiting and Assisting Developers Based on Their In-IDE Behaviors. *Proceedings of the ACM on Software Engineering* 3, FSE (2026), 435–458. doi:10.1145/3797147

[27] Shan Lu, Soyeon Park, Eunsoo Seo, and Yuanyuan Zhou. 2008. Learning from Mistakes: A Comprehensive Study on Real World Concurrency Bug Characteristics. In *Proceedings of the 13th International Conference on Architectural Support for Programming Languages and Operating Systems*. Association for Computing Machinery, New York, NY, USA, 329–339. doi:10.1145/1346281.1346323

[28] Zhuang Luo, Yichuan Li, Zexing Xu, Kyumin Lee, and S. Rasoul Etesami. 2025. FaithfulPersona: Balancing Faithfulness and Personalization in Code Explanations through Self-Critique. In *Findings of the Association for Computational Linguistics: NAACL 2025*. Association for Computational Linguistics, Albuquerque, New Mexico, 930–944. doi:10.18653/v1/2025.findings-naacl.53

[29] Xinbei Ma, Yeyun Gong, Pengcheng He, Hai Zhao, and Nan Duan. 2023. Query Rewriting in Retrieval-Augmented Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Singapore, 5303–5315. doi:10.18653/v1/2023.emnlp-main.322

[30] Meta. 2024. Llama-3.1-8B-Instruct: Model Card. Retrieved October 3, 2026 from https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct

[31] Fangwen Mu, Junjie Wang, Lin Shi, Song Wang, Shoubin Li, and Qing Wang. 2026. ExpeRepair: Dual-Memory Enhanced LLM-Based Repository-Level Program Repair. *Proceedings of the ACM on Software Engineering* 3, FSE (2026), 3954–3976. doi:10.1145/3808181

[32] OpenAI. n.d. Codex. Retrieved October 3, 2026 from https://developers.openai.com/learn/codex

[33] OpenAI. n.d. GPT-5.6 Sol Model. Retrieved October 3, 2026 from https://developers.openai.com/api/docs/models/gpt-5.6-sol

[34] OpenAI. n.d. GPT-5.6 Terra Model. Retrieved October 3, 2026 from https://developers.openai.com/api/docs/models/gpt-5.6-terra

[35] Matheus Paixão, Anderson Uchôa, Ana Carla Bibiano, Daniel Oliveira, Alessandro Garcia, Jens Krinke, and Emilio Arvonio. 2020. Behind the Intents: An In-depth Empirical Study on Software Refactoring in Modern Code Review. In *Proceedings of the 17th International Conference on Mining Software Repositories*. Association for Computing Machinery, Seoul, Republic of Korea, 125–136. doi:10.1145/3379597.3387475

[36] Philip I. Pavlik, Jr., Hao Cen, and Kenneth R. Koedinger. 2009. Performance Factors Analysis—A New Alternative to Knowledge Tracing. In *Proceedings of the 14th International Conference on Artificial Intelligence in Education (Frontiers in Artificial Intelligence and Applications, Vol. 200)*. IOS Press, Amsterdam, The Netherlands, 531–538. doi:10.3233/978-1-60750-028-5-531

[37] Weihan Peng, Yuling Shi, Yingwei Ma, Longfei Yun, Beijun Shen, and Xiaodong Gu. 2026. DeepRepoQA: Code Repository Question Answering with Deep Agent Exploration. arXiv:2608.24221 [cs.SE] doi:10.48550/arXiv.2608.24221

[38] Weihan Peng, Yuling Shi, Yuhang Wang, Xinyun Zhang, Beijun Shen, and Xiaodong Gu. 2025. SWE-QA: Can Language Models Answer Repository-level Code Questions? arXiv:2509.14635v1 [cs.CL] https://arxiv.org/abs/2509.14635v1 Version 1.

[39] Weihan Peng, Yuling Shi, Yuhang Wang, Xinyun Zhang, Beijun Shen, and Xiaodong Gu. 2026. SWE-QA: Can Language Models Answer Repository-level Code Questions?. In *Findings of the Association for Computational Linguistics: ACL 2026*. Association for Computational Linguistics, San Diego, California, United States, 8230–8245. doi:10.18653/v1/2026.findings-acl.402

[40] Steven I. Ross, Fernando Martinez, Stephanie Houde, Michael Muller, and Justin D. Weisz. 2023. The Programmer's Assistant: Conversational Interaction with a Large Language Model for Software Development. In *Proceedings of the 28th International Conference on Intelligent User Interfaces*. Association for Computing Machinery, New York, NY, USA, 491–514. doi:10.1145/3581641.3584037

[41] Jonathan Sillito, Gail C. Murphy, and Kris De Volder. 2006. Questions Programmers Ask During Software Evolution Tasks. In *Proceedings of the 14th ACM SIGSOFT International Symposium on Foundations of Software Engineering*. Association for Computing Machinery, New York, NY, USA, 23–34. doi:10.1145/1181775.1181779

[42] Leopoldo Todisco, Antonio Della Porta, Stefano Lambiase, and Fabio Palomba. 2026. TARS: A Theory-of-Mind Agent for Personalized In-IDE Code Comprehension. arXiv:2607.15948 [cs.SE] doi:10.48550/arXiv.2607.15948 Accepted to the Tool Demonstration and Data Showcase Track of the 2026 IEEE International Conference on Software Maintenance and Evolution.

[43] Svitlana Vakulenko, Shayne Longpre, Zhucheng Tu, and Raviteja Anantha. 2021. Question Rewriting for Conversational Question Answering. In *Proceedings of the 14th ACM International Conference on Web Search and Data Mining*. Association for Computing Machinery, Virtual Event, Israel, 355–363. doi:10.1145/3437963.3441748

[44] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Daniel Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *The Thirteenth International Conference on Learning Representations*. International Conference on Learning Representations, Singapore, 65882–65919. https://proceedings.iclr.cc/paper_files/paper/2025/hash/a4b6ad6b48850c0c331d1259fc66a69c-Abstract-Conference.html

[45] Xin Xia, Lingfeng Bao, David Lo, Pavneet Singh Kochhar, Ahmed E. Hassan, and Zhenchang Xing. 2017. What Do Developers Search for on the Web? *Empirical Software Engineering* 22, 6 (2017), 3149–3185. doi:10.1007/s10664-017-9514-4

[46] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., Vancouver, Canada, 50528–50652. doi:10.52202/079017-1601

[47] Xu Yang, Jiayuan Zhou, Michael Pacheco, Wenhan Zhu, Pengfei He, Shaowei Wang, Kui Liu, and Ruiqi Pan. 2026. Lingxi: Repository-Level Issue Resolution Framework Enhanced by Procedural Knowledge Guided Scaling. In *Proceedings of the 35th ACM SIGSOFT International Symposium on Software Testing and Analysis*. Association for Computing Machinery, Oakland, CA, USA. https://conf.researchr.org/details/issta-2026/issta-2026-research-papers/179/Lingxi-Repository-Level-Issue-Resolution-Framework-Enhanced-by-Procedural-Knowledge- Forthcoming.

[48] Z.ai. 2026. GLM-5.3-Flash. Retrieved October 3, 2026 from https://huggingface.co/zai-org/GLM-5.3-Flash

[49] Jing Zhang, Lianghong Guo, Yanlin Wang, Terry Yue Zhuo, Yong Wang, Mingwei Liu, Jiachi Chen, Ensheng Shi, Yuchi Ma, Hongyu Zhang, and Zibin Zheng. 2026. From Conversation to Evaluation: Benchmarking LLMs on Development Knowledge via SimpleDevQA. In *Findings of the Association for Computational Linguistics: ACL 2026*. Association for Computational Linguistics, San Diego, California, United States, 37649–37663. doi:10.18653/v1/2026.findings-acl.1877

[50] Jiuang Zhao, Zitian Yang, Li Zhang, Xiaoli Lian, Donghao Yang, and Xin Tan. 2024. DRMiner: Extracting Latent Design Rationale from Jira Issue Logs. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. Association for Computing Machinery, Sacramento, CA, USA, 468–480. doi:10.1145/3691620.3695019

[51] Wenhan Zhu, Haoxiang Zhang, Ahmed E. Hassan, and Michael W. Godfrey. 2022. An Empirical Study of Question Discussions on Stack Overflow. *Empirical Software Engineering* 27, 6, Article 148 (2022), 25 pages. doi:10.1007/s10664-022-10180-z

[52] Thomas Zimmermann, Andreas Zeller, Peter Weißgerber, and Stephan Diehl. 2005. Mining Version Histories to Guide Software Changes. *IEEE Transactions on Software Engineering* 31, 6 (2005), 429–445. doi:10.1109/TSE.2005.72